



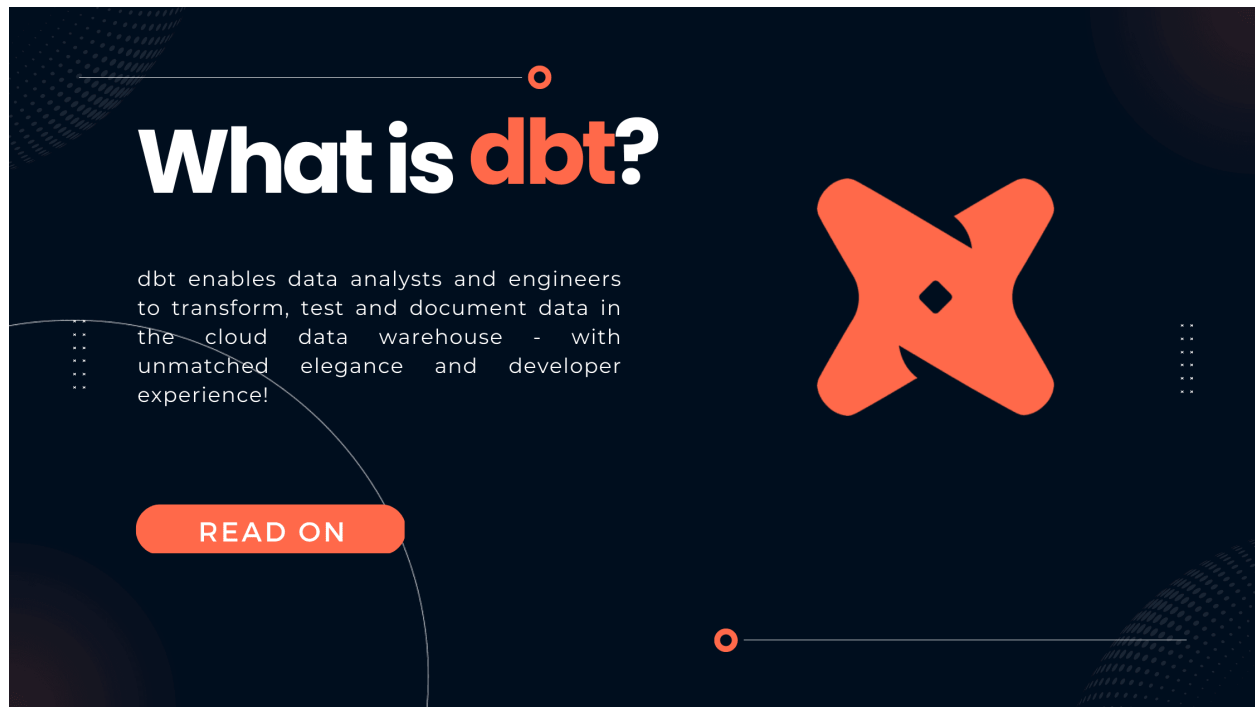
Data Engineering

SETUP DBT



What is dbt?

🔧 dbt is the tool that turns messy SQL scripts into organized data pipelines and lets analysts feel like software engineers. 😊



dbt (Data Build Tool) is an open-source command-line tool that enables data analysts and engineers to transform, test, and document data in a data warehouse. It follows the concept of "dbt projects," which are sets of SQL queries and configurations organized into models. dbt provides a structured workflow for managing the development, testing, and deployment of these models.

🍏 **SETUP dbt in Mac**

Step:1 Install Homebrew (if not already installed):

- Open Terminal.
- Run the following command:

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

Step:2 Install Python 3 (if not already installed):

- Open Terminal.
- Run the following command using Homebrew:

```
brew install python@3.9
```

Step:3 Install dbt:

- Open Terminal.
- Run the following command to install dbt and its dependencies:

```
pip install dbt
```

Step: 4 Verify the installation:

- Open Terminal.
- Run the following command to check if dbt is installed successfully:

```
dbt --version
```

You should see the version number of dbt printed in the terminal.

1. Set up your dbt project:

- Create a new directory for your dbt project (e.g., my_dbt_project) and navigate to it in the Terminal.
- Run the following command to initialize the dbt project:

```
dbt init my_dbt_project
```

This will create the necessary project structure and files.

2. Configure your dbt profiles:

- Open the profiles.yml file in your dbt project directory using a text editor.
- Update the default profile or add new profiles to connect to your database(s). Here's an example configuration for a PostgreSQL database:

```
my_postgres_db:
  target: dev
  outputs:
    dev:
      type: postgres
      host: localhost
      port: 5432
      user: my_username
      password: my_password
      dbname: my_database
      schema: public
      threads: 1
```

Customize the values according to your specific database connection details.

3. Test the connection:

- Open Terminal.
- Run the following command to check if dbt can connect to your database using the configured profile:

dbt debug

- You should see debug information about your database connection if it is successful.

You have now set up dbt on your macOS machine. You can start building and running your dbt models by following the dbt documentation and using the dbt CLI commands.

SETUP dbt in Windows

Before setting up dbt, ensure you have Git and Python installed.

Step 1: Install Git and Python

Before setting up dbt, ensure you have Git and Python installed.

Installing Git

First, download Git from the official website: [Git for Windows](#). Then, run the installer and:

- Choose "Git from the command line and also from 3rd-party software."
- Select "Use the default Windows console window."
- Leave other options as default and complete the installation.

Then, verify the installation by running the following in Command Prompt. If you see a version number, Git is successfully installed.

```
git --version
```



```
Microsoft Windows [Version 10.0.26200.8655]
(c) Microsoft Corporation. All rights reserved.

C:\Users\craft>git --version
git version 2.49.0.windows.1

C:\Users\craft>
```

Installing Python

First, download Python from python.org. Then, run the installer and check the box for "Add Python to PATH." Then, complete the installation. Once you've completed the installation, verify by running the following in Command Prompt. If you see a version number, Python is ready to go!

```
python --version
```



```
Microsoft Windows [Version 10.0.26200.8655]
(c) Microsoft Corporation. All rights reserved.

C:\Users\craft>python --version
Python 3.14.6

C:\Users\craft>
```

Step 2: Set Up Your Project Folder

Now, it's time to create a folder where your dbt project will live. First, open Command Prompt (Win + R, type cmd, and hit Enter). Then, run the following commands to create and enter your project folder:

```
mkdir my_dbt_project
```

```
cd my_dbt_project
```

Now you're inside your project folder!

Step 3: Set Up a Virtual Environment

Creating a virtual environment keeps dbt isolated from other Python projects. First, run the following command inside your project folder. This creates a (venv) folder inside your project.

```
python -m venv venv
```

Then, activate the virtual environment. If you see (venv) at the beginning of your command line, you're inside the virtual environment.

```
venv\Scripts\activate
```

Step 4: Initialize Git *(Optional but Recommended)*

If you're using Git for version control, initialize a repository. First, start a new Git repository:

```
git init
```

OR clone an existing dbt project:

```
git clone <repository-url>
```

```
cd <project-folder-name>
```

Now, your project is under version control!

Step 5: Set Up the profiles.yml File

The profiles.yml file tells dbt how to connect to your data warehouse (like Snowflake, BigQuery, or Postgres). This file is typically found in **C:\Users <your_username>\.dbt\profiles.yml**.

Locate or Create the File

First, check if the file exists by running the following command.

```
cd %USERPROFILE%\.dbt && dir
```

If profiles.yml is missing, create it:

```
mkdir %USERPROFILE%\.dbt
```

```
notepad %USERPROFILE%\.dbt\profiles.yml
```

Example Configuration (For Snowflake)

Copy and paste this into profiles.yml, replacing placeholders with your actual Snowflake credentials.

```
my_dbt_project:

  outputs:

    dev:

      type: snowflake

      account: <account_name>

      user: <username>

      password: <password>

      role: <role>

      database: <database_name>

      warehouse: <warehouse_name>

      schema: <schema_name>

  target: dev
```

Step 6: Install dbt

Now, it's time to install dbt inside your virtual environment. Ensure your virtual environment is active by running the following. (venv) should be visible in the prompt.

```
pip install dbt-core dbt-snowflake
```

Replace dbt-snowflake with dbt-postgres, dbt-bigquery, etc., depending on your data warehouse.

Step 7: Verify your dbt Installation

Check if dbt is installed correctly:

```
dbt --version
```

You should see the installed dbt version and plugins. If it works, congratulations—dbt is set up!

Troubleshooting Tips

If you run into issues, here are some quick solutions.

Virtual Environment Not Activating

If `venv\Scripts\activate` doesn't work, try:

call `venv\Scripts\activate`

Or, if using PowerShell, run:

`venv\Scripts\Activate.ps1`

If you get a security error, enable script execution with:

`Set-ExecutionPolicy Unrestricted -Scope Process`

pip Command Not Found

If pip isn't recognized, install it manually:

`python -m ensurepip --default-pip`

dbt Command Not Found

Ensure your virtual environment is activated before running dbt.

SETUP dbt on VS Code

Now that dbt is installed, you can start transforming your data! Some next steps to explore:

For developing our data models we are going to use VSCode - as it provides awesome extensions which make working with dbt fun and exciting.

1. If not already done, [install vscode](#).
2. Set up the folder structure, as follows
 1. Add a `.gitignore` file with the following content

```
1target/  
2dbt_modules/  
3dbt_packages/  
4logs/  
5env/  
6env/  
7test.env  
8__pycache__  
9.vscode
```



```
10export.json
```

2. Run **dbt init** and follow the screen. The init application might ask different questions, depending on which database adapter you are going to use. In this example, I'm using BigQuery as an example (make sure you have installed the appropriate adapter - see the **pip install** steps above)

```
1dbt init
2# This is the result
317:06:52 Running with dbt=1.4.5
4Enter a name for your project (letters, digits, underscore): waffle_shop
5Which database would you like to use?
6[1] bigquery
7(Dont see the one you want? https://docs.getdbt.com/docs/available-
adapters)
8Enter a number: 1
9[1] oauth
10[2] service_account
11Desired authentication method option (enter a number): 2
12keyfile (/path/to/bigquery/keyfile.json): /tmp/google_key.json
13project (GCP project id): devopsandmore
14dataset (the name of your dbt dataset): dbt_devops
15threads (1 or more): 8
16job_execution_timeout_seconds [300]: 300
17[1] US
18[2] EU
19Desired location option (enter a number): 2
2017:07:25 Profile waffle_shop written to /home/andreas/.dbt/profiles.yml
using targets profile_template.yml and your supplied values. Run 'dbt debu
g' to validate the connection.
2117:07:25
22Your new dbt project "waffle_shop" was created!
23For more information on how to configure the profiles.yml file,
24please consult the dbt documentation here:
25https://docs.getdbt.com/docs/configure-your-profile
26One more thing:
27Need help? Don't hesitate to reach out to us via GitHub issues or on Sla
ck:
28https://community.getdbt.com/
29Happy modeling!
```

Finishing this installer will create the following folder structure

```
1.
```

```
2 |— logs
3 |   |— dbt.log
4 |— waffle_shop
5 |— README.md
6 |— analyses
7 |— dbt_project.yml
8 |— macros
9 |— models
10 |   |— example
11 |   |— my_first_dbt_model.sql
12 |   |— my_second_dbt_model.sql
13 |   |— schema.yml
14 |— seeds
15 |— snapshots
16 |— tests
```

- **logs:** This folder contains the logs of the dbt init operation. You can delete it.
 - **waffle_shop:** This is the main dbt project folder. All your model sql files, description files and settings are located here. The name of the folder is the name of the dbt project you entered during **dbt init**.
 - **README.md:** Readme file. Feel free to change.
 - **analyses:** Folder which might contain [dbt analyses](#). Despite it's name, analyses are not the main thing in dbt - "models" are
 - **dbt_project.yaml:** Project settings file
 - **macros:** Folder to store [dbt macros](#)
 - **models:** This is the folder you will spend most of your time in. It contains all [dbt models](#) - as a reminder, dbt models are "just" sql files, enhanced with jinja template code.
 - **example:** Delete this folder
 - **seeds:** Folder which might contain [dbt seeds](#)
 - **snapshots:** Folder which might contain [dbt snapshots](#)
 - **tests:** Folder containing [custom dbt tests](#)
3. Install vscode-dbt. This adds support for the dbt syntax (dbt-sql, jinja, yaml, markdown)

4. Install sqlfluff

```
pip install sqlfluff
```

5. Install the [sqlfluff vscode extension](#)
6. Add a file **.sqlfluff** to the toplevel folder, with the following content:

```
1[sqlfluff]
2# Choose the sql dialect to use
3dialect = bigquery
4# Use jinja templater instead of dbt, as the dbt templater is slow
5# and does not work with bigquery dialect
6templater = jinja
7# Optional: Exclude some rules
8# Find all rules here: https://docs.sqlfluff.com/en/stable/rules.html
9exclude_rules = L034, L032, L024, L015, L008, L001
10ignore_templated_areas = True
11large_file_skip_byte_limit = 200000
12[sqlfluff:templater:jinja]
13# Make dbt default macros available like `ref`
14apply_dbt_builtins = true
15# Load custom macros from the macro directory
16load_macros_from_path = ./macros/
17# Allow mocking dbt_utils and other packages through
18# python files placed in this dir
19library_path = ./sqlfluff_libs/
20# Mock variables that are used in dbt
21[sqlfluff:templater:jinja:context]
22target = prod
23[sqlfluff:rules]
24tab_space_size = 4
25max_line_length = 100
```

7. Add a file **.sqlfluffignore**, with following lines:

```
1target/
2dbt_modules/
3dbt_packages/
4macros/
5tests/
```

8. Add a folder **.vscode** at the same folder level as **logs** and **waffle_shop**.
9. Add a file **settings.json** inside the **.vscode** folder, having the following content:

```

1{
2// change this to where your python dbt virtual environment python binary
  is located
3"python.python.defaultInterpreterPath": "./.dbtenv/bin/python",
4"files.associations": {
5// associate all sql files in the folder with jinja sql
6"*.sql": "jinja-sql",
7// optional: don't format models in `target/` dir.
8// This disables syntax highlighting in the target folder
9// This helps to very quickly identify, that you are not in a model file,
10// but a compiled target file.
11"**/target/**": ""
12},
13"editor.quickSuggestions": {
14"strings": true
15},
16// run "whereis sqlfluff" to find the executable path of your sqlfluff i
  nstallation
17"sqlfluff.executablePath": "/path/to/dbt-venv/bin/sqlfluff",
18"sqlfluff.dialect": "bigquery",
19"sqlfluff.linter.run": "onSave",
20"sqlfluff.experimental.format.executeInTerminal": true,
21"editor.formatOnSave": false,
22"dbt.queryLimit": 1000,
23"editor.rulers": [100]
24}

```

10. Install the [dbt power user vscode extension](#)

If you follow the steps above you'll get a basic dbt setup and vscode as sort of dbt IDE. The developer experience is quite nice and you have a development system at hand to start working with dbt.

What's Next?

Now that dbt is installed, you can start transforming your data! Some next steps to explore:

- Running dbt commands like dbt run and dbt test.
- Learning about dbt models, tests, and macros.

- Setting up dbt Cloud for team collaboration.

Congrats on making it through the setup! 🎉